

CSP-J/S 模拟（一）

2026年6月27日

说明：J组不需要做附加题，S组需要做J的所有题目及附加题。

一、单项选择题

- 共 15 题，每题 2 分，共计 30 分
 - 每题有且仅有一个正确选项
1. 将C++源代码转化为机器代码的具体步骤是（ ）。
A. 预处理、编译、链接
B. 编辑、编译、运行
C. 编译、运行、调试
D. 编译、链接、运行
 2. 存储一张 4096×2160 像素的 32 位真彩色图片需要的空间最接近（ ）。
A. 24.84 MB
B. 29.70 MB
C. 31.64 MB
D. 33.75 MB
 3. 算式 $(25_8 + 1011_2) \times 2_{16}$ 的结果以十进制表示为（ ）。
A. 50
B. 52
C. 54
D. 64
 4. 某个栈一开始为空，依次执行以下操作：

```
push(a)
push(b)
pop()
push(c)
push(d)
pop()
pop()
push(e)
```

则该栈的容量至少是（ ）。

- A. 2
- B. 3
- C. 4

D. 5

5. 链表相较于数组的优势在于（ ）。

- A. 随机访问速度快
- B. 插入与删除操作效率高
- C. 内存连续分配，缓存友好
- D. 不需要额外存储指针

6. 二叉树的前序遍历为 F, C, A, D, B, E ，中序遍历为 A, C, B, D, F, E ，则后序遍历是（ ）。

- A. A, B, C, D, E, F
- B. A, B, D, C, E, F
- C. A, C, B, E, D, F
- D. A, B, C, E, D, F

7. 一张有向图的顶点编号为 A, B, C, D ，邻接矩阵如下所示，则合法的拓扑排序是（ ）。

	A	B	C	D
A	[0, 1, 0, 0]			
B	[0, 0, 1, 1]			
C	[0, 0, 0, 1]			
D	[0, 0, 0, 0]			

- A. A, B, C, D
- B. A, C, B, D
- C. A, B, D, C
- D. A, C, D, B

8. 后缀表达式 $3\ 4\ *\ 2\ 5\ +\ -\ 6\ *$ 的求值结果是（ ）。

- A. -30
- B. 30
- C. 42
- D. -42

9. 将 6 本不同的书分给 3 人，每人至少 1 本，有（ ）种分法。

- A. 540
- B. 360
- C. 90
- D. 120

10. 当 $x > 0$ 时，表达式 $(x \ \& \ (x-1)) == 0$ 用于判断（ ）。

- A. x 是否为偶数
- B. x 是否为 2 的幂
- C. x 是否为 0
- D. x 的二进制表示中是否存在两个或以上的 1

11. 对 100 个整数进行冒泡排序，最少需要比较（ ）次。
- A. 99
 - B. 4950
 - C. 100
 - D. 5050

12. 函数 $f(n)$ 定义如下：

```
int f(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    else {  
        return n * f(n-2);  
    }  
}
```

- 则 $f(6)$ 的返回值是（ ）。
- A. 24
 - B. 120
 - C. 720
 - D. 48

13. 四种字符的出现频率如下：

字符	A	B	C	D
出现频率	40%	30%	20%	10%

- 则 A 的哈夫曼编码长度是（ ）。
- A. 1
 - B. 2
 - C. 3
 - D. 4

14. 三人过桥的时间分别为 1 分钟、3 分钟、6 分钟。最多只允许两人同时过桥，两人过桥时，按单人最慢的时间计算。三人只有一盏灯，过桥时必须持有灯。则三人全部过桥的最短总时间为（ ）。
- A. 9 分钟
 - B. 10 分钟
 - C. 11 分钟
 - D. 12 分钟

15. 关于差分数组的性质，下列说法正确的是（ ）。
- A. 差分数组是原数组的前缀和。
 - B. 差分数组的第 i 项表示原数组中第 i 项和第 $i - 1$ 项的差值。
 - C. 差分数组的第 i 项表示原数组中第 i 项和第 $i + 2$ 项的差值。
 - D. 差分数组无法通过前缀和恢复成原数组。

二、阅读程序

- 判断题 1 分，选择题 3 分，共计 37 分
- 判断题正确填 T，错误填 F；

第1题

```
#include<iostream>
int main() {
    int a, b;
    std::cin >> a >> b;
    int s = 0;
    int c = 0;
    while (a > 0 || b > 0) {
        int x = a % 10;
        int y = b % 10;
        a /= 10;
        b /= 10;
        if (x + y + c >= 10) {
            c = 1;
            s++;
        }
        else {
            c = 0;
        }
    }
    std::cout << s << "\n";
}
```

判断题

16. 当输入为 1 1 时，程序的输出为 2。
17. 若输入的两个数都是不超过 99999 的正整数，则输出的结果可能大于 5。
18. 输出的 s 应该永远小于或等于 a 且永远小于或等于 b 。

选择题

19. 当输入为 998244353 31415926 时，程序的最终输出为（ ）。

- A. 1029660279
- B. 4
- C. 5
- D. 966828427

20. 当输入为 307506234 692493766 时，程序的最终输出为（ ）。

- A. 384987532
- B. 8
- C. 996998991
- D. 9

第2题

```
#include<iostream>
int main() {
    int d;
    std::cin >> d;
    char s[1000];
    int size = 0;
    char c;
    while (std::cin >> c) {
        while (size > 0 and d > 0) {
            char top = s[size - 1];
            if (top < c) {
                d--;
                size--;
            }
            else {
                break;
            }
        }
        s[size++] = c;
    }
    while (d > 0) {
        d--;
        size--;
    }
    for (int i = 0; i < size; ++i) {
        std::cout << s[i];
    }
}
```

判断题

21. 把 `while (size > 0 and d > 0)` 改成 `while (size >= 0 and d > 0)` 后程序功能保持不变。
22. 代码 `s[size++] = c;` 可修改为 `size+=1; s[size] = c;`
23. 若循环 `while (std::cin >> c)` 执行了 10 次，当 `d` 的输入值超过 10 时，程序会崩溃。

选择题

24. 当输入为 3 31415926 时，则程序的输出为（ ）。
- A. 314
 - B. 926
 - C. 45926
 - D. 11526
25. 记循环 `while (std::cin >> c)` 执行了 n 次，该程序的时间复杂度为（ ）。
- A. $\Theta(n)$
 - B. $\Theta(nd)$
 - C. $\Theta(n \log n)$
 - D. $\Theta(n + d)$
26. 这段程序的功能是移除输入的 d 个字符，使得剩余字符串（ ）。
- A. 字典序最小
 - B. 字典序最大
 - C. 字符种类数最多
 - D. 下降子序列最长

第3题

```
#include<iostream>
int choose[20];
int dfs(int m, int n) {
    if (m == n) {
        bool flag = true;
        for (int i = 0; i + 1 < n; ++i) {
            if (choose[i] && choose[i+1]) {
                flag = false;
            }
        }
        if (flag) {
            return 1;
        }
        else {
            return 0;
        }
    }
    else {
```

```

        choose[m] = true;
        int pick = dfs(m+1, n);
        choose[m] = false;
        int drop = dfs(m+1, n);
        return pick + drop;
    }
}

int fib(int n) {
    int f[21];
    f[0] = 1;
    f[1] = 2;
    for (int i = 2; i <= n; ++i) {
        f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}

int main() {
    int n;
    std::cin >> n;
    std::cout << dfs(0, n) << " ";
    std::cout << fib(n) << "\n";
}

```

判断题

27. 当 n 的输入值介于 0 到 20 之间时, $\text{dfs}(0, n)$ 的返回值永远等于 $\text{fib}(n)$ 。
28. 程序结束后, `choose` 数组的各项元素均等于 `false`。
29. `fib` 的时间复杂度是 $\Theta(n)$ 。
30. `dfs` 的时间复杂度是 $\Theta(n^2)$ 。

选择题

31. `dfs` 函数采用的算法思想是 ()。
 - A. 迭代
 - B. 分治
 - C. 枚举
 - D. 动态规划
32. 当输入为 4 时, 则输出为 ()。
 - A. 7 8
 - B. 8 7
 - C. 8 8
 - D. 7 7

33. dfs 函数的主要功能是 ()
- A. 统计长度为 n 的序列的子段数量
 - B. 统计长度为 n 的序列中不含连续 1 的二进制序列数量
 - C. 统计关于 n 的一些组合数的数量
 - D. 统计所有长度为 n 的二进制序列数量
34. 在 dfs 函数中, choose 数组的主要作用是什么?
- A. 无任何作用
 - B. 记录当前递归路径的选择状态
 - C. 作为动态规划的记忆表
 - D. 存储待输出的序列

三、完善程序

- 单选题, 每小题 3 分, 共计 33分

第1题

给定 n 个数字 $a_1, a_2, \dots, a_n$, 所谓有序对是指从 1 到 n 中挑出两个下标 i 与 j 并要求 $i < j$, 然后将 a_i 与 a_j 组成一个有序的序对 (a_i, a_j) 。

请统计, 能从序列中挑选出多少种不相等的数对? 数对 (x, y) 与 (p, q) 称之为不相等, 是指 $x \neq p$ 或 $y \neq q$ 。

```
#include<iostream>
int a[100];
int c[100];
bool present[100];
int main() {
    int n;
    std::cin >> n;
    int num = 0;
    long long pair = 0;
    for (int i = 0; i < n; ++i) {
        std::cin >> a[i];
        pair += ____ (1) ____;
        pair -= c[____ (2) ____] ;
        ____ (3) ____ = num;
        if (____ (4) ____) {
            present[a[i]] = true;
            ____ (5) ____;
        }
    }
    std::cout << pair << "\n";
}
```


35. (1) 处应填 ()。

- A. num
- B. i
- C. a[i]
- D. 1

36. (2) 处应填 ()。

- A. num
- B. i
- C. a[i]
- D. 1

37. (3) 处应填 ()。

- A. c[i]
- B. c[a[i]]
- C. a[i]
- D. a[c[i]]

38. (4) 处应填 ()。

- A. present[a[i]]
- B. present[c[i]]
- C. !present[c[i]]
- D. !present[a[i]]

39. (5) 处应填 ()。

- A. num+=c[i]
- B. num+=a[i]
- C. present[c[i]] = true
- D. num++

第2题

给定一个由 $n \times m$ 个方格组成的网络，每个方格内有一个正整数，其中第 i 行第 j 列为 $a_{i,j}$ 。

可以使用任意多块 1×2 的骨牌覆盖网格上的数字，每块骨牌不得重叠，也不能越过网格的边界。

请问应该如何摆放骨牌，使得没有被覆盖的数字的异或之和达到最大。注意不覆盖任何骨牌也是一种选择。

```
#include<iostream>
int a[20][20];
bool covered[20][20];
int n, m;
int solve(int x, int y, int sum) {
    if (y == m) {
        return ____ (2) ____ ;
    }
}
```

```

}
if (____(3)____) {
    return sum;
}
int D = 0;
if (covered[x][y]) {
    D = solve(x, y+1, sum);
} else {
    D = ____ (4) ____ ;
}
int V = 0;
int H = 0;
if (!covered[x][y] && y+1 < m && !covered[x][y+1]) {
    covered[x][y] = covered[x][y+1] = true;
    V = ____ (5) ____;
    covered[x][y] = covered[x][y+1] = false;
}
if (!covered[x][y] && x+1 < n) {
    covered[x][y] = covered[x+1][y] = true;
    H = ____ (6) ____;
    covered[x][y] = covered[x+1][y] = false;
}
return std::max(D, std::max(H, V));
}

int main() {
    std::cin >> n >> m;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            std::cin >> a[i][j];
        }
    }
    std::cout << ____ (1) ____ << "\n";
}

```

40. (1) 处应填 ()。

- A. solve(0, 0, 0)
- B. solve(0, 0, 1)
- C. solve(1, 1, 0)
- D. solve(1, 1, 1)

41. (2) 处应填 ()。

- A. solve(x+1, 0, sum)
- B. solve(x+1, y, sum)
- C. solve(x, y+1, sum)

D.solve(x+1, y+1, sum)

42.(3)处应填 ()。

A.x > n

B.x == n

C.x == n - 1

D.x < n

43.(4)处应填 ()。

A.solve(x, y+1, sum ^ a[x][y])

B.solve(x+1, y, sum ^ a[x][y])

C.solve(x, y+1, a[x][y])

D.solve(x+1, y, sum)

44.(5)处应填 ()。

A.solve(x, y+1, sum ^ a[x][y])

B.solve(x+1, y, sum ^ a[x][y])

C.solve(x, y+1, sum)

D.solve(x+1, y, sum)

45.(6)处应填 ()。

A.solve(x, y+1, sum ^ a[x][y])

B.solve(x+1, y, sum ^ a[x][y])

C.solve(x, y+1, sum)

D.solve(x+1, y, sum)

四、附加题

第1题

```
struct flat_map
{
    struct {
        int key;
        int value;
    } bucket[65536];
    int size = 0;

    int get(int key) {
        int begin = 0;
        int end = size;
        while (begin < end) {
            int mid = begin + (end - begin) / 2;
            if (bucket[mid].key < key) {
                begin = mid + 1;
            }
        }
    }
}
```

```

        else {
            end = mid;
        }
    }
    if (begin < size and bucket[begin].key == key) {
        return bucket[begin].value;
    }
    else {
        return 0;
    }
}

void put(int key, int value) {
    int moved = 0;
    while (moved < size) {
        int test = size - moved - 1;
        if (key < bucket[test].key) {
            bucket[test + 1] = bucket[test];
            moved++;
        }
        else {
            break;
        }
    }
    bucket[size - moved] = {key, value};
    size++;
}
};

```

判断题

46. flat_map 实现了一种关系型容器（ ）。
47. flat_map 对按照键的大小顺序，将键与值配对，存储到了一块连续的内存序列里（ ）。
48. 当调用 get(k) 后发现 k 不存在，flat_map 会为 k 分配一块内存并将它对应的值置为 0（ ）。

选择题

49. 记 s 表示容器的大小 size，调用 get(key) 的最坏时间复杂度为（ ）。
- A. $\Theta(s)$
- B. $\Theta(\log s)$
- C. $\Theta(s \log s)$
- D. $\Theta(s^2)$

50. 记 s 表示容器的大小 `size`，调用 `put(key, value)` 的最坏时间复杂度为 ()。
- A. $\Theta(s)$
 - B. $\Theta(\log s)$
 - C. $\Theta(s \log s)$
 - D. $\Theta(s^2)$
51. 以下说法错误的是 ()。
- A. `flat_map` 的优点是插入数据时移动数据少。
 - B. `flat_map` 的优点是数据紧凑排列，空间使用效率高。
 - C. `flat_map` 的优点是查找数据的效率高。
 - D. `flat_map` 的优点是代码紧凑，实现简短。
52. 若调用 `put(k, v)` 时已经存在相同的键 k 时，以下哪一种处理策略最符合上述代码的逻辑 ()。
- A. 熔断 (Breaker)：抛出异常，向系统报告错误
 - B. 回滚 (Rollback)：不做任何修改，撤销 `put` 操作
 - C. 覆盖 (Rewrite)：将键所对应的老值覆盖成新值
 - D. 忽略 (Ignore)：对有可能出错的操作置之不理

第2题

一个序列的次大值，是指这个序列中排名第二大的数，如果序列只有一个数，则次大数定义为 0。

给定一个 1 到 n 的排列 $p_1, p_2, \dots, p_n$ ，请统计这个排列的所有连续子序列的次大数之和。

定义 $\max_2(a_i, a_{i+1}, \dots, a_j)$ 表示从 a_i 开始到 a_j 结束的次大数，题目所要求出的是：

$$\sum_{1 \leq i \leq j \leq n} \max_2(a_i, a_{i+1}, \dots, a_j)$$

函数 `solve` 用于解决这个问题。

```
int q[maxn];
int prev[maxn];
int next[maxn];
long long solve(int n, int p[])
{
    for (int i = 1; i <= n; ++i)
    {
        ____ (1) ____ ;
    }
    p[0] = q[0] = prev[0] = 0;
    p[n+1] = q[n+1] = next[n+1] = n+1;
    for (int i = 1; i <= n; ++i)
    {
        int num = ____ (2) ____;
        int prev_num = p[i-1];
        int next_num = p[i+1];
```

```

    prev[num] = ____ (3) ____;
    next[num] = ____ (4) ____;
}
long long sum = 0;
for (int num = 1; num <= n; ++num) {
    int prev_num = prev[num];
    int prev_prev_num = prev[prev_num];

    int next_num = next[num];
    int next_next_num = next[next_num];

    sum += (long long) num * ____ (5) ____ * (q[next_num] - q[num]);
    sum += (long long) num * (q[num] - q[prev_num]) * ____ (6) ____ ;

    ____ (7) ____ = prev_num;
    ____ (8) ____ = next_num;
}
return sum;
}

```

53. (1)处应填 ()。

- A. $q[p[i]] = i$
- B. $q[i] = p[i]$
- C. $q[i] = 1$
- D. $q[i] = i$

54. (2)(3)(4)处应填 ()。

- A. $p[i]$, $prev_num$, $next_num$
- B. $q[i]$, $next_num$, $prev_num$
- C. $p[i]$, $prev_num$, $next_num$
- D. $q[i]$, $next_num$, $prev_num$

55. (5)处应填 ()。

- A. $(q[prev_num] - q[prev_prev_num])$
- B. $(q[next_next_num] - q[prev_num])$
- C. $(q[next_num] - q[prev_prev_num])$
- D. $(q[next_next_num] - q[next_num])$

56. (6)处应填 ()。

- A. $(q[prev_num] - q[prev_prev_num])$
- B. $(q[next_next_num] - q[prev_num])$
- C. $(q[next_num] - q[prev_prev_num])$
- D. $(q[next_next_num] - q[next_num])$

57.(7)(8)处应填 ()。

- A. prev[prev_num] , next[next_num]
- B. prev[next_num] , next[prev_num]
- C. next[prev_num] , prev[next_num]
- D. next[next_num] , prev[prev_num]